Basic (your first working tool) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `first_tool_call_practice.py`: this is the exact primitive every agent in this entire curriculum is built from — a model deciding to call a function instead of answering directly. Seeing it work once, standalone, before combining it with anything else, is what makes the Build Task's `run_with_tools()` obvious instead of magic. **If not:** the first time you'd see a tool call requested and executed would be inside a bigger harness already juggling 2-3 tools, making it harder to tell which part is "the round trip" and which part is everything else.

Solution

```
# first_tool_call_practice.py
from langchain_core.runnables import Runnable
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b

def get_model_with_tools() -> Runnable:
    # how: .bind_tools() returns a new Runnable, not a ChatOpenAI
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([add])

def main() -> None:
    model_with_tools = get_model_with_tools()
    response = model_with_tools.invoke("What's 5 + 7?")

    if not response.tool_calls:
        # when: even an obvious math question, the model can still just
        # answer in text instead of calling a tool — always check first.
        print("The model answered directly, without calling a tool.")
        print(response.content)
        return

    for call in response.tool_calls:
        if call["name"] == "add":
```

```
            # how: .invoke() on the tool itself, not a plain function call —
            # @tool wraps add into a Runnable too.
            result = add.invoke(call["args"])
            print(f"Called add({call['args']}) -> {result}")
        else:
            print(f"Unexpected tool requested: {call['name']}")

if __name__ == "__main__":
    main()
```

Expected output:

```
Called add({'a': 5, 'b': 7}) -> 12
```